

Introduction to JavaScript

JavaScript is a lightweight, interpreted programming language designed for creating network-centric applications. It is a versatile programming language commonly used to create interactive effects within web browsers. It's one of the core technologies of the World Wide Web, alongside HTML and CSS.

It is widely supported cross platform object oriented client side scripting language developed by Netscape. When incorporated with HTML, it makes the HTML programming a dynamic one. It is available with all the browsers.

Some Key points to remember while writing the code :

- Java Script is case sensitive.
- Statements in JavaScript are generally followed by a semicolon
- JavaScript ignores spaces, tabs, and newlines that appear in the programs



Why Learn JavaScript?

- **Interactivity:** Enhance user engagement on websites.
- **Versatility:** Used in web development, server-side programming, game development, and more.
- **Demand:** High demand for JavaScript developers in the job market.

Features :

- Displays messages to the user in the browser's status line or in alert boxes
- Validates the contents of the form
- Animates the images with events
- Creates ad banners
- Detects the browser in use and perform advanced functions
- Detects installed plug-ins(Active-X controls)
- Modifies all or part of the webpage
- Displays and interact with user input.

- JavaScript is a dynamic or loosely-typed language because a variable can hold value of any data type at any point of time.

Limitations :

- Client-side JavaScript does not allow the reading or writing of files. This has been kept for security reason.
- JavaScript can be prone to security risks like Cross-Site Scripting (XSS), Cross-Site request Forgery (CSRF), and injection attacks, as it is executed on the client-side and can be manipulated by malicious actors.
- JavaScript doesn't have any multithreading or multiprocessor capabilities.
- While JavaScript is fast for many tasks, it may not be as performant as lower-level languages like C or C++ for resource-intensive operations (e.g., complex data processing, graphics rendering).

Setting Up Your Environment

To start coding in JavaScript, you'll need:

1. **A Web Browser:** Modern browsers like Chrome, Firefox, or Edge.
2. **A Text Editor or IDE:** Options include Visual Studio Code, Sublime Text, or Notepad++.

Placement of Javascript in HTML Page

It can be placed and used in an HTML document in several ways, depending on the desired behavior and where you want the script to run.

1. **Inline JavaScript :** Inline JavaScript refers to embedding JavaScript directly within an HTML element using the onclick, onmouseover, onchange, or other event attributes. This method is generally used for small scripts tied to specific events.

Example:

```
<button onclick="alert('Hello, World!')">Click Me</button>
```

In this example, the JavaScript alert function is executed when the button is clicked.

2. **Internal JavaScript :** Internal JavaScript is placed within the <script> tags in the <head> or <body> section of an HTML page. This method is useful for including JavaScript that is specific to the page, and it allows the script to be written directly in the HTML document.

Example

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Internal JavaScript Example</title>
  <script>
    function showMessage() {
      alert('Hello from internal JavaScript!');
    }
  </script>
</head>
<body>
  <button onclick="showMessage()">Click Me</button>
</body>
</html>

```

3. **External JavaScript** : External JavaScript involves writing the JavaScript code in a separate .js file and linking to it from within the HTML document using the <script> tag. This method is highly recommended for larger projects or when the same script needs to be reused across multiple HTML pages.

Example

Myscript.js

```

function showMessage() {
  alert('Hello from external JavaScript!');
}

```

Mytest.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>External JavaScript Example</title>
</head>
<body>
  <button onclick="showMessage()">Click Me</button>

  <script src="myscript.js"></script>
</body>
</html>

```

Script tag:

The <script> tag in HTML is essential for embedding and linking JavaScript code in a web page. It plays a key role in enabling dynamic behavior and interactivity on websites. The <script> tag allows the browser to understand and execute JavaScript code within an HTML document. The script tag is usually placed within the < Head >, however, there is no restriction of its placement in any part of the page.

The main attributes of <script> tag are

- **Src:** The src attribute specifies the URL of an external JavaScript file to be included and executed. This is the most common use of the <script> tag to link to external scripts.

```
<script src="script.js"></script>
```

- **Type:** The type attribute specifies the scripting language used in the script. In modern HTML5, the type attribute is optional, as JavaScript is the default.

```
<script type="text/javascript">  
    // JavaScript code  
</script>
```

Basics of JavaScript Syntax

JavaScript syntax is the set of rules that define how a JavaScript program is constructed.

Comments

- Single-line comment: `// This is a comment`
- Multi-line comment:

```
/*  
This is a  
multi-line comment  
*/
```

- HTML Style comment (Works but not recommended): The closing for comment html is ignored by javascript.

```
<!-- A single-line comment.  
<!-- Hello there  
    var z = 9;  
-->
```

Here, only Hello there will be commented, but var z = 9 will be considered as normal code.

Case Sensitivity

JavaScript is case-sensitive. For example, myVariable and myvariable are different.

Semicolons

Semicolons are used to separate statements. While they are optional in many cases, it's good practice to include them.

Variables and Data Types

Variables store data values that can be changed later. A variable is a container which has a name and values are assigned to it. We use variables to hold information that may change from one moment to the next while a program is running.

Naming Rules:

- Variable names must begin with a letter or an underscore
- Variable names must not include spaces
- JavaScript is case-sensitive
- Reserved keywords can not be used as variable name

Reserved Keywords in Javascript

abstract	else	instanceof	switch	boolean
enum	int	synchronized	break	export
interface	this	byte	extends	long
throw	case	false	native	throws
catch	final	new	transient	char
finally	null	true	class	float
package	try	const	for	private
typeof	continue	function	protected	var
debugger	goto	public	void	default
if	return	volatile	delete	implements
short	while	do	import	static

abstract	else	instanceof	switch	boolean
with	double	in	super	

Storing a value in a variable is called variable initialization. You can do variable initialization at the time of variable creation or at a later point in time when you need that variable.

Declaring Variables

- **var:** Older way to declare variables. The var is the oldest keyword to declare a variable in JavaScript. It has the Global scope or function scope. This means:
 - If you create a variable outside of a function, you can use it anywhere in your code.
 - If you create a variable inside a function, you can only use it within that function.
- **let:** The let keyword is an improved version of the var keyword. It is introduced in the ES6 or EcmaScript 2015. These variables have the block scope. It can't be accessible outside the particular code block ({block}).
 - The variable is only available inside the block {} where you define it.
- **const:** Used for variables that won't change.
- **Without any keyword:** Used to just assign value to some variable without having var, let or const. It behaves like var by default.

Example Definition:

```
<script type = "text/javascript">
    let age;
    var name;
</script>
```

Example Initialization:

```
<script type = "text/javascript">
    let age = 20;
    var name = 'Ram';
    const id = 22445;
    someVar = 'hello';
</script>
```

Note: Most of the people are confused about the scope in var and let. Lets understand this through one example:

```
<script>
  function myFunction() {
    if (true) {
      var varVariable = 10;
      let letVariable = 20;

      console.log(varVariable); // Output: 10(accessible within the if block)
      console.log(letVariable); // Output: 20(accessible within the if block)
    }

    console.log(varVariable); // Output: 10(varVariable is accessible here)
    //console.log(letVariable); // ReferenceError:letVariable is not defined
  }

  myFunction();
</script>
```

In the above example, since varVariable is of function scope, even if it is defined inside if block, it can be accessed anywhere within that function. But in case of let, since it is of block scoped, it can not be accessed outside the block it is declared.

Data Types

A data type, in programming, is a classification that specifies which type of value a variable has and what type of mathematical, relational or logical operations can be applied to it without causing an error.

JavaScript is an untyped (or dynamically typed) language, meaning that you don't need to specify the type of a variable when declaring it, data types still exist to describe what kind of data a variable holds. The type of a variable is determined at runtime, and the same variable can hold different types of values at different times.

Datatypes can be classified to two broad categories ,

- **Primitive:** A primitive data type is a data type that store value. These datatypes do not have methods.
- **Non-Primitive:** A non-primitive data type is a data type that is an object and has methods. They store references.

Here are the main data types in JavaScript:

- **String:** Text data ('Hello', "World")

- **Number:** Numeric data (42, 3.14)
- **Boolean:** Logical data (true, false)
- **Null:** No value (null)
- **Undefined:** A declared variable without a value (undefined)
- **Object:** Complex data structures
- **Date:** Dates and times
- **Array:** For storing ordered collections of values. Example: [1, 2, 3, 4]

The blue ones (String to Undefined) are primitive and red ones (Object to Array) are non primitive data types.

Operators

Operators perform operations on variables and values. Operators are a type of command. They perform operations on variables and/or literals and produce a result.

Javascript supports following types of operators.

1. Arithmetic Operators
2. Comparison Operators
3. Logical (or Relational) Operators
4. Bitwise Operators
5. Assignment Operators
6. Conditional (or ternary) Operators

Arithmetic Operators

Assume variable A holds 10 and variable B holds 20 then

Operator	Description	Example
+	Adds two operands	A + B will give 30
-	Subtracts second operand from the first	A - B will give -10
*	Multiply both operands	A * B will give 200
/	Divide numerator by denominator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	B % A will give 0
++	Increment operator, increases integer value by one	A++ will give 11
--	Decrement operator, decreases integer value by one	A-- will give 9

Comparison Operators

Assume variable A holds 10 and variable B holds 20 and C holds "10" then

Operator	Description	Example
===	Checks if the value of two operands are equal or not in their value and type, if yes then condition becomes true.	(A===B) is not true. (A===C) is not true.
==	Checks if the value of two operands are equal or not in their value, if yes then condition becomes true.	(A==B) is not true. (A==C) is true.
!=	Checks if the value of two operands are equal or not, if values are not equal then condition becomes true.	(A!=B) is true. (A!=C) is false.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A>=B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A<=B) is true.

Logical (or Relational) Operators

Assume variable A holds true and variable B holds true then

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non zero then then condition becomes true.	(A && B) is true.
	Called Logical OR Operator. If any of the two operands are non zero then then condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is false.

Assignment Operators

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand	$C = A + B$ will assign value of $A + B$ into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand	$C /= A$ is equivalent to $C = C / A$
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand	$C \% = A$ is equivalent to $C = C \% A$

Bitwise Operator

Assume variable A holds 2 and variable B holds 3 then

Operator	Description	Example
&	Called Bitwise AND operator. It performs a Boolean AND operation on each bit of its integer arguments.	$(A \& B)$ is 2 .
	Called Bitwise OR Operator. It performs a Boolean OR operation on each bit of its integer arguments.	$(A B)$ is 3.
^	Called Bitwise XOR Operator. It performs a Boolean exclusive OR operation on each bit of its integer arguments. Exclusive OR means that either operand one is true or operand two is true, but not both.	$(A \wedge B)$ is 1.
~	Called Bitwise NOT Operator. It is a unary operator and operates by reversing all bits in the operand.	$(\sim B)$ is -4
<<	Called Bitwise Shift Left Operator. It moves all bits in its first operand to the left by the number of places specified in the second operand. New bits are filled with zeros. Shifting a value left by one position is equivalent to multiplying by 2, shifting two positions is equivalent to multiplying by 4, etc.	$(A << 1)$ is 4.

>>	Called Bitwise Shift Right with Sign Operator. It moves all bits in its first operand to the right by the number of places specified in the second operand. The bits filled in on the left depend on the sign bit of the original operand, in order to preserve the sign of the result. If the first operand is positive, the result has zeros placed in the high bits; if the first operand is negative, the result has ones placed in the high bits. Shifting a value right one place is equivalent to dividing by 2 (discarding the remainder), shifting right two places is equivalent to integer division by 4, and so on.	(A>>1) is 1.
>>>	Called Bitwise Shift Right with Zero Operator. This operator is just like the >> operator, except that the bits shifted in on the left are always zero,	(A>>>1) is 1.

Conditional(ternary) Operator (?:)

The conditional operator first evaluates an expression for a true or false value and then executes one of the two given statements depending upon the result of the evaluation.

Condition ?X:Y

Here if condition is true then the value is X otherwise value is Y

Conditional Statements

Conditional statements perform different actions based on different conditions.

if Statement

```
if (condition) {
  // code to execute if condition is true
}
```

Example

```
let score = 85;
if (score >= 40) {
  console.log('You Passed');
}
```

if...else Statement

```
if (condition) {
  // code if true
} else {
  // code if false
}
```

Example

```
let score = 85;
if (score >= 40) {
    console.log('You Passed');
} else {
    console.log('You Failed');
}
```

if...else if...else Statement

```
if (condition1) {
    // code if condition1 is true
} else if (condition2) {
    // code if condition2 is true
} else {
    // code if both conditions are false
}
```

Example

```
let score = 85;
if (score >= 90) {
    console.log('Grade A');
} else if (score >= 80) {
    console.log('Grade B');
} else {
    console.log('Grade C');
}
```

switch Statement

A switch statement is used to execute different statement based on different conditions. It provides a better alternative than a long series of if. else if . statements.

```
switch(expression) {
case value1:
    // code
    break;
case value2:
    // code
    break;
```

```
default:
    // code
}
```

Example

```
let day = 3;
let dayName;

switch (day) {
  case 1:
    dayName = "Monday";
    break;
  case 2:
    dayName = "Tuesday";
    break;
  case 3:
    dayName = "Wednesday";
    break;
  case 4:
    dayName = "Thursday";
    break;
  case 5:
    dayName = "Friday";
    break;
  case 6:
    dayName = "Saturday";
    break;
  case 7:
    dayName = "Sunday";
    break;
  default:
    dayName = "Invalid day";
}
console.log(dayName); // Output: Wednesday
```

Loops

A loop statement checks to see if some condition is true, and if that condition is true, it executes a chunk of code. After the code is executed, the condition is checked again. If it is true, the process starts over again; if it is false, the loop stops and the rest of the code

continues along. A loop is a sequence of instructions that is repeated until a certain condition is reached.

Different Types of loops:

- **Entry Controlled loops:** In this type of loops the test condition is tested before entering the loop body. For Loop and While Loop are entry controlled loops.
- **Exit Controlled Loops:** In this type of loops the test condition is tested or evaluated at the end of loop body. Therefore, the loop body will execute at least once, irrespective of whether the test condition is true or false. do — while loop is exit controlled loop

for Loop

```
for (initialization; condition; increment) { // code to execute
}
```

Example:

```
for (let i = 0; i < 5; i++) {
  console.log('Iteration number ' + i);
}
```

for-in Loop

The for...in loop is used to iterate over the enumerable properties of an object or the indexes of an array.

```
for (let key in object) {
  // Code to execute
}
```

Example:

```
const numbers = [10, 20, 30, 40];
for (let index in numbers) {
  console.log(index + ": " + numbers[index]);
}
```

while Loop

```
while (condition) {
  // code to execute
}
```

Example:

```
let i = 0;
while (i < 5) {
    console.log('Iteration number ' + i);
    i++;
}
```

do...while Loop

```
do {
    // code to execute
} while (condition);
```

Example:

```
let i = 0;
do {
    console.log('Iteration number ' + i);
    i++;
} while (i < 5);
```

Functions

Functions are reusable blocks of code that perform a specific task. Function is a named block of statements which can be executed again and again simply by writing its name and can return some value. It is useful in making a program modular and understandable. JavaScript functions do not require you to specify the type of data they will return.

Declaring a Function

```
function functionName(parameters) {
    // code to execute
}
```

Calling a Function

```
functionName(arguments);
```

Example

Function returning values

```
function greet(name) {
    return 'Hello, ' + name + '!';
}
```

```
let message = greet('Alice');  
console.log(message); // Outputs: Hello, Alice!
```

Function without returning values

```
function greet(name) {  
    message = 'Hello, ' + name + '!';  
    Console.log(message);  
}  
  
greet('Alice'); // Outputs: Hello, Alice!
```

JavaScript Popup boxes

In JavaScript, a popup box (or dialog box) is a small, temporary window that appears on the screen, usually on top of the main web page content. It's used to display information to the user or to get input from them.

Popups can be used for a variety of purposes, such as:

- Alerts: Displaying simple messages to the user (e.g., warnings, notifications).
- Confirmations: Asking the user to confirm an action (e.g., "Are you sure you want to delete this?").
- Prompts: Getting input from the user (e.g., asking for their name).

JavaScript has three built-in functions for creating simple popups:

- **alert():** Displays a simple alert box with a message and an "OK" button. It's modal, meaning it blocks further execution until the user clicks "OK". It's primarily used for simple notifications or warnings.
- **confirm():** Displays a confirmation dialog with a message and two buttons: "OK" and "Cancel". It returns a boolean value (true if the user clicks "OK", false if they click "Cancel").
- **prompt():** Displays a dialog box with a message, a text input field, and "OK" and "Cancel" buttons. It returns the value entered by the user (or null if the user clicks "Cancel").

Example:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <title>Popup Box Example</title>  
</head>  
<body>
```



```
<button type="button" onclick="showAlert()">Alert</button>
<button type="button" onclick="showConfirm()">Confirm</button>
<button type="button" onclick="showPrompt()">Prompt</button>
</body>
<script>
    function showAlert(){
        alert("This is alert box");
    }
    function showConfirm(){
        var Val = confirm("Do you want to continue ?");
        if (Val == true) {
            console.log("Please continue flow");
            return true;
        } else {
            console.log("please stop the flow");
            return false;
        }
    }
    function showPrompt(){
        var val = prompt("Enter your name : ", "Please enter your name");
        console.log("You entered : " + val);
        alert("You entered : " + val);
    }
</script>
</html>
```

Objects and Properties

An object in JavaScript is a collection of properties (key-value pairs). The key is always a string, and the value can be of any data type: a number, string, array, another object, function, etc.

A property is one of the key-value pairs within an object. The key is often referred to as the "property name," and the associated value is the "property value."

JavaScript is template based not class based. Here, we don't create class to get the object. But, we directly create objects.

There are 3 ways to create objects.

1. By object literal
2. By creating instance of Object directly (using new keyword)
3. By using an object constructor (using new keyword)

Create Object by object literal

The syntax of creating object using object literal is given below:

object={property1:value1,property2:value2.....propertyN:valueN}

```
person = {
  name: 'John',
  age: 30,
  city: 'New York',
  show: function() {
    console.log("Hello, my name is " + this.name + " and I am " +
      this.age + " years old.");
  }
};
```

By creating instance of Object directly (using new keyword)

The syntax of creating object directly is given below. Here, new keyword is used to create object

var objectname=new Object();

```
const person = new Object();
person.name = 'John';
person.age = 30;
person.city = 'New York';
Person.show = function() {
  console.log("Hello, my name is " + this.name + " and I am " +
    this.age + " years old.");
}
```

By using an object constructor (using new keyword)

Here, you need to create function with arguments. Each argument value can be assigned in the current object by using this keyword. The this keyword refers to the current object.

```
function Employee(name, age, city) {
  this.name = name;
  this.age = age;
  this.city= city;
  this.show = function() {
    console.log("Hello, my name is " + this.name + " and I am " +
      this.age + " years old.");
  }
}
```

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Object Example</title>
</head>
<body>

  <h1>Person Information</h1>

  <script>
    // JavaScript object
    person = {
      name: 'John',
      age: 30,
      city: 'New York'
    };

    document.write("<p>Name: " + person.name + "</p>");
    document.write("<p>Age: " + person.age + "</p>");
    document.write("<p>City: " + person.city + "</p>");
  </script>

</body>
</html>
```

Accessing Object Properties

- **Dot Notation:** person.name
- **Bracket Notation:** person['name']

Adding/Updating Properties

```
person.job = 'Developer';
person['age'] = 31;
```

Deleting Properties

```
delete object.property;
delete object[property];
delete person['age'];
```

```
Delete person.age;
```

Complete Example (For CREATE, UPDATE and DELETE properties)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Object Property Manipulation</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
    }
    input, button {
      margin: 5px 0;
      padding: 8px;
      font-size: 14px;
    }
    label {
      font-size: 16px;
    }
    .output {
      margin-top: 20px;
    }
  </style>
</head>
<body>

  <h1>Object Property Manipulation</h1>

  <h3>Initial Object</h3>
  <pre id="personObject">person = { name: 'John', age: 30, city: 'New
York' }</pre>

  <h3>Create/Update a Property</h3>
  <label for="propName">Property Name:</label>
  <input type="text" id="propName" placeholder="Enter property
name"><br>

  <label for="propValue">Property Value:</label>
```

```

    <input type="text" id="propValue" placeholder="Enter property
value"><br>

    <button onclick="createOrUpdateProperty()">Create/Update
Property</button>

    <h3>Delete a Property</h3>
    <label for="deletePropName">Property Name to Delete:</label>
    <input type="text" id="deletePropName" placeholder="Enter property
name to delete"><br>

    <button onclick="deleteProperty()">Delete Property</button>

    <div class="output">
        <h3>Current Object:</h3>
        <pre id="currentObject"></pre>
    </div>

    <script>
        // Initial object
        let person = {
            name: 'John',
            age: 30,
            city: 'New York'
        };

        // Function to update the displayed object
        function updateDisplay() {
            document.getElementById('personObject').textContent=
JSON.stringify(person, null, 2);
            document.getElementById('currentObject').textContent=
JSON.stringify(person, null, 2);
        }

        // Function to create or update a property
        function createOrUpdateProperty() {
            const propName = document.getElementById('propName').value;
            const propValue = document.getElementById('propValue').value;

            if (propName && propValue !== '') {
                person[propName] = propValue;
                alert(`Property "${propName}" has been added/updated.`);
            }
        }
    </script>

```

```

        updateDisplay();
    } else {
        alert('Please enter both a property name and a value.');
```

```
    }
```

```
    // Clear input fields
```

```
    document.getElementById('propName').value = '';
```

```
    document.getElementById('propValue').value = '';
```

```
    }
```

```
    // Function to delete a property
```

```
    function deleteProperty() {
```

```
        const deletePropName =
```

```
document.getElementById('deletePropName').value;
```

```
        if (deletePropName && person.hasOwnProperty(deletePropName)) {
```

```
            delete person[deletePropName];
```

```
            alert(`Property "${deletePropName}" has been deleted.`);
```

```
            updateDisplay();
```

```
        } else {
```

```
            alert('Property not found or input is empty.');
```

```
        }
```

```
        // Clear input field
```

```
        document.getElementById('deletePropName').value = '';
```

```
    }
```

```
    // Initial display of object
```

```
    updateDisplay();
```

```
</script>
```

```
</body>
```

```
</html>
```

Arrays

Arrays are used to store multiple values in a single variable. Array is an object that represents a collection of similar type of elements.

There are 3 ways to construct/create array in JavaScript

1. By array literal
2. By creating instance of Array directly (using new keyword)
3. By using an Array constructor (using new keyword)

Creating Arrays

By array literal

```
let fruits = ['Apple', 'Banana', 'Cherry'];
```

By creating instance of Array directly

```
var fruits = new Array();
```

```
fruits[0] = "Apple";
```

```
fruits[1] = "Banana";
```

```
fruits[2] = "Cherry";
```

By using an Array Constructor

```
var fruits = new Array('Apple', 'Banana', 'Cherry');
```

Accessing Array Elements

```
let firstFruit = fruits[0]; // 'Apple'
```

Example:

```
<html>
<body>
<script>
var fruits=["Apple","Banana","Cherry"];
for (i=0;i<fruits.length;i++)
{
document.write(fruits[i] + "<br/>");
}
</script>
</body>
</html>
```

Example2:

```
<html>
<body>
<script>
    var fruits = new Array();
    fruits[0] = "Apple";
    fruits[1] = "Banana";
    fruits[2] = "Cherry";
    for (i=0;i<fruits.length;i++)
    {
        document.write(fruits[i] + "<br/>");
    }
</script>
</body>
</html>
```

Array Methods

- **push():** Adds an element to the end
- **pop():** Removes the last element
- **shift():** Removes the first element
- **unshift():** Adds an element to the beginning
- **length:** Returns the number of elements
- **toString():** Converts the array into a comma-separated string of its elements.
- **sort(Object):** Sorts the array's elements in place. It accepts an optional comparison function (Object) to define the sorting order.
- **splice(Number, 0, any type):** Inserts an element ("any type") at a specified index (Number) in the array. No elements are removed as the second argument(number of items to be removed) is 0.
- **splice(Number, Number):** Removes a specified number of elements (second Number) starting from a given index (first Number). It returns the removed elements as a new array.

Example:

```
fruits.push('Date');
console.log(fruits); // ['Apple', 'Banana', 'Cherry', 'Date']
```

Example:

```
// ArrayMethods.js
```



```

var numArray = [1,3,8,4,9,7,6,2,5];

// Sort in ascending order
numArray.sort(
    function compare (first, second) { return first - second; }
);

// numArray.toString(): 1,2,3,4,5,6,7,8,9

numArray.splice(2, 0, 2.5);
// numArray.toString(): 1,2,2.5,3,4,5,6,7,8,9

// output of following: 5,6,7
window.alert(numArray.splice(5,3).toString());

// numArray.toString(): 1,2,2.5,3,4,8,9
window.alert(numArray.toString());

```

Built in Objects in Javascript

JavaScript has several built-in or core language objects. These built-in objects are available regardless of window content and operate independently of whatever page your browser has loaded. [They are available in any JavaScript environment, whether it's a browser, Node.js, or any other JavaScript runtime.](#) These built-in objects provide a rich set of tools for working with data, performing calculations, manipulating text, handling dates and times, and much more.

Here are some of the most commonly used built-in objects in JavaScript:

- **String Object:** provides methods for working with strings
- **Math Object:** provides mathematical functions and constants
- **Number Object:** provides methods for working with numbers
- **Array Object:** provides methods for working with arrays
- **Date Object:** provides methods for working with dates and times
- **RegExp Object:** provides methods for working with regular expressions
- **Boolean Object:** provides methods for working with Boolean values
- **Object Object:** provides methods for working with objects
- **Function Object:** provides methods for working with functions
- **Error Object:** provides methods for working with errors

Additionally there is **Window object** which can be called as global object. The window object in JavaScript is a special object that represents the current browser window or tab in which your script is running. It's not a built-in JavaScript object (like Object or Array), but rather a host object provided by the browser environment. It's the top-level object in the browser's object hierarchy and provides access to a wide range of browser-related functionalities.

Window Object

The global object of JavaScript in the web browser is the window object. It means that all variables and functions declared globally with the var keyword become the properties and methods of the window object. It represents the current browser window and provides methods for manipulating the window, such as opening or closing it, changing its size or position, and navigating to a new URL. The window object is supported by all browsers. It represents the browser's window. All global JavaScript objects, functions, and variables automatically become members of the window object. Global variables are properties of the window object. Global functions are methods of the window object. Even the document object (of the HTML DOM) is a property of the window object.

The window object provides access to different browser-related functionalities. They can be listed as.

- **DOM (Document Object Model):** The document object, which is a property of window, represents the HTML document loaded in the browser. It allows you to access and manipulate the structure, content, and style of the web page. For example: `window.document.getElementById("myElement")`.
- **Browser History:** The window.history object provides access to the browser's history, allowing you to navigate back and forward.
- **Location Information:** The window.location object provides information about the current URL and allows you to change it.
- **Screen Information:** The window.screen object provides information about the user's screen.
- **Navigator Information:** The window.navigator object provides information about the browser itself (e.g., user agent, platform).
- **Popup Boxes:** Methods like `alert()`, `confirm()`, and `prompt()` allow you to display popup boxes to interact with the user. `Window.alert()`, `window.confirm()`, `window.prompt()`.

- **Frames:** If the page contains frames (using the <frame> or <iframe> tags), you can access them through the window.frames property.

Apart from browser-related functionalities, window object also provides methods for managing browser windows themselves.

- **window.open():** Opens a new window or tab.
- **window.close():** Closes the current window (though this is often restricted by browsers for security reasons).
- **window.moveTo():** Moves the window to a new position.
- **window.resizeTo():** Resizes the window.

You can use all the window functions without window. prefix. This is because the window object is the global object in browsers, and its properties and methods are automatically available in the global scope.

Example:

// These are equivalent in a browser:

```
alert("Hello!");           // Implicit window.
window.alert("Hello!");    // Explicit window.
```

```
console.log(document.title); // Implicit window.
console.log(window.document.title); // Explicit window.
```

```
setTimeout(myFunction, 1000); // Implicit window.
window.setTimeout(myFunction, 1000); // Explicit window.
```

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Windows Object Example</title>
</head>
<body>
  <button type="button" onclick="windowsFunction()">Windows Function
  1</button>
```

```

    <button type="button" onclick="windowsFunctions()">Windows Function
2</button>
</body>
<script>

    function windowsFunction(){
        //shows the alert
        window.alert("Opened from windows, now trying to open window");

        //opens the url in new tab
        window.open("https://www.google.com");

        //closes the current window
        window.close();
    }

    function windowsFunctions(){
        // Get the current URL
        console.log("Location: "+window.location.href);

        // Get the hostname (domain)
        console.log("Domain: "+window.location.hostname);

        // Get the pathname (part of the URL after the domain)
        console.log("Pathname: "+window.location.pathname);

        // Get the user agent string
        console.log("User Agent: "+window.navigator.userAgent);

        // Get the platform (OS)
        console.log("Platform: "+window.navigator.platform);
    }
</script>
</html>

```

String Object:

The String object is used to represent and manipulate a sequence of characters. Strings are useful for holding data that can be represented in text form. Some of the most-used operations on strings are to check their length, to build and concatenate them using

the + and += string operators, checking for the existence or location of substrings with the indexOf() method, or extracting substrings with the substring() method.

Creating Strings.

Strings can be created in different ways.

- **Using string literals:** To create literal strings, you use either single quotes (') or double quotes (") like this:
- **Using the String() constructor**

Example:

```
const string1 = "A string primitive"; // Using string literal
const string2 = new String("A String object"); //Using string()
constructor.
```

String Interpolation

You can create strings using Template Literals. Template literals allow you to embed expressions within backticks (`) for dynamic string creation, making it more readable and versatile.

Example:

```
let name = 'Ram';
let str = `Hello ${s1} `;

console.log(str); //Hello Ram
```

String Object Properties

Property	Description
length	Returns the length of a string.
prototype	Allows you to add new properties and methods to an String object.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>String Example</title>
```

```

</head>
<script>

    // Adding a custom method to String.prototype to check for vowels
    String.prototype.hasVowel = function() {
        // Define the vowels
        const vowels = "aeiouAEIOU";

        // Loop through each character of the string
        for (let i = 0; i < this.length; i++) {
            // If a vowel is found, return true
            if (vowels.indexOf(this[i]) !== -1) {
                return true;
            }
        }
        // If no vowels were found, return false
        return false;
    };

    // Creating String
    let myString1 = "Hello";
    let myString2 = new String("Sky");

    console.log(myString1.hasVowel()); // Output: true (contains vowels)
    console.log(myString2.hasVowel()); // Output: false (no vowels)
    console.log(myString2.length); // Output: 3

</script>
</html>

```

String Object Methods

Method	Description
charAt()	Returns the character at the specified index.
charCodeAt()	Returns the Unicode of the character at the specified index.
concat()	Joins two or more strings, and returns a new string.
endsWith()	Checks whether a string ends with a specified substring.

<code>fromCharCode()</code>	Converts Unicode values to characters.
<code>includes()</code>	Checks whether a string contains the specified substring.
<code>indexOf()</code>	Returns the index of the first occurrence of the specified value in a string.
<code>lastIndexOf()</code>	Returns the index of the last occurrence of the specified value in a string.
<code>localeCompare()</code>	Compares two strings in the current locale.
<code>match()</code>	Matches a string against a regular expression, and returns an array of all matches.
<code>repeat()</code>	Returns a new string which contains the specified number of copies of the original string.
<code>replace()</code>	Replaces the occurrences of a string or pattern inside a string with another string, and return a new string without modifying the original string.
<code>search()</code>	Searches a string against a regular expression, and returns the index of the first match.
<code>slice()</code>	Extracts a portion of a string and returns it as a new string.
<code>split()</code>	Splits a string into an array of substrings.
<code>startsWith()</code>	Checks whether a string begins with a specified substring.
<code>substr()</code>	Extracts the part of a string between the start index and a number of characters after it.
<code>substring()</code>	Extracts the part of a string between the start and end indexes.
<code>toLocaleLowerCase()</code>	Converts a string to lowercase letters, according to host machine's current locale.
<code>toLocaleUpperCase()</code>	Converts a string to uppercase letters, according to host machine's current locale.
<code>toLowerCase()</code>	Converts a string to lowercase letters.
<code>toString()</code>	Returns a string representing the specified object.
<code>toUpperCase()</code>	Converts a string to uppercase letters.
<code>trim()</code>	Removes whitespace from both ends of a string.

valueOf()	Returns the primitive value of a String object.
-----------	---

Example :

```
const str = "Hello, World!";

// Accessing characters
str.charAt(0); // "H"
str.charCodeAt(0); // 72
str[0]; // "H"

// Combining strings
str.concat(" Welcome!"); // "Hello, World! Welcome!"
"Hello".concat(", ", "World", "!"); // "Hello, World!"

// Searching strings
str.indexOf("World"); // 7
str.lastIndexOf("o"); // 8
str.startsWith("Hello"); // true
str.endsWith("!"); // true
str.includes("orl"); // true
str.search("World"); // 7

// Extracting substrings
str.slice(7, 12); // "World"
str.substring(7, 12); // "World"
str.substr(7, 5); // "World"

// Modifying strings
str.replace("World", "Universe"); // "Hello, Universe!"
str.replaceAll("o", "0"); // "Hell0, W0rld!"
str.toUpperCase(); // "HELLO, WORLD!"
str.toLowerCase(); // "hello, world!"
str.trim(); // "Hello, World!" (removes whitespace from both ends)
" Hello ".trim(); // "Hello"
str.trimStart(); // "Hello, World!" (removes whitespace from the start)
" Hello ".trimStart(); // "Hello "
str.trimEnd(); // "Hello, World!" (removes whitespace from the end)
" Hello ".trimEnd(); // " Hello"
str.padStart(15, "*"); // "****Hello, World!"
```



```

str.padEnd(15, "*"); // "Hello, World!***"

// Splitting strings
str.split(","); // ["Hello", " World!"]
"apple,banana,cherry".split(","); // ["apple", "banana", "cherry"]

// String length
str.length; // 13

// Repeating strings
str.repeat(2); // "Hello, World!Hello, World!"

// Template literals (using backticks)
`Hello, ${"World"}!`; // "Hello, World!"

// Converting to string
String(123); // "123"
(123).toString(); // "123"

```

Date Object:

In JavaScript, the Date object is a built-in object that allows you to work with dates and times. It provides a variety of methods to handle dates, compare them, manipulate them, and format them.

The Date object in JavaScript represents a single point in time. It provides various functionalities to work with dates and times like creating a Date object in several ways, and it allows you to perform operations like extracting parts of the date (year, month, day), comparing dates, and manipulating dates.

You can use different variant of Date constructor to create date object.

```

Date()
Date(dateString)
Date(year,month)
Date(year,month,day)
Date(year,month,day,hours)
Date(year,month,day,hours,minutes)
Date(year,month,day,hours,minutes,seconds)
Date(year,month,day,hours,minutes,seconds,ms)
Date(milliseconds)

```

- `new Date()` creates a date object with the current date and time

```
const d = new Date();
```

- `new Date(date string)` creates a date object from a date string.

```
const d = new Date("October 13, 2014 11:13:00");  
const d = new Date("2022-03-25");
```

- `new Date(year, month, ...)` creates a date object with a specified date and time.

- ◆ 7 numbers specify year, month, day, hour, minute, second, and millisecond (in that order)

```
const d = new Date(2018, 11, 24, 10, 33, 30, 0);
```

Notes:

- JavaScript counts months from 0 to 11:
- January = 0.
- December = 11.
- Specifying a month higher than 11, will not result in an error but add the overflow to the next year.

```
const d = new Date(2018, 15, 24, 10, 33, 30);
```

Is the same as:

```
const d = new Date(2019, 3, 24, 10, 33, 30);
```

- Specifying a day higher than max, will not result in an error but add the overflow to the next month:

```
const d = new Date(2018, 5, 35, 10, 33, 30);
```

Is the same as:

```
const d = new Date(2018, 6, 5, 10, 33, 30);
```

- ◆ 6 numbers specify year, month, day, hour, minute, second.

```
const d = new Date(2018, 11, 24, 10, 33, 30);
```

- ◆ 5 numbers specify year, month, day, hour, and minute:

```
const d = new Date(2018, 11, 24, 10, 33);
```

- ◆ 4 numbers specify year, month, day, and hour:

```
const d = new Date(2018, 11, 24, 10);
```

- ◆ 3 numbers specify year, month, and day:

```
const d = new Date(2018, 11, 24);
```

- ◆ 2 numbers specify year and month:

```
const d = new Date(2018, 11);
```

You cannot omit month. If you supply only one parameter it will be treated as milliseconds from epoch.

```
const d = new Date(2018);
```

NOTE: JavaScript stores dates as number of milliseconds since January 01, 1970. (Epoch)

Date Object Methods.

Name	Description
<code>getDate</code>	Use to get the day of the month of a given date according to local time.
<code>getDay</code>	Use to get the day of the week of a given date according to local time.
<code>getFullYear</code>	Use to get the year of a given date according to local time.
<code>getHours</code>	Use to get the hour of a given date according to local time.
<code>getMilliseconds</code>	Use to get the milliseconds from the current time according to local time.
<code>getMinutes</code>	Use to get the minutes from the current time according to local time.
<code>getMonth</code>	Use to get the month of a given date according to local time
<code>getSeconds</code>	Use to get the seconds from the current time according to local time.
<code>getTime</code>	Use to get the milliseconds of a given date according to local time.
<code>getTimezoneOffset</code>	Use to get the difference between local time and Greenwich Mean Time (GMT).
<code>getUTCDate</code>	Returns the day (date) of the month of a given date according to Coordinated Universal Time (UTC).
<code>getUTCDay</code>	Returns the day of the week in the specified date according to universal time(UTC).
<code>getUTCFullYear</code>	Returns the year of a given date according to universal time(UTC).
<code>getUTCHours</code>	Returns the hours of given date according to universal time(UTC).
<code>getUTCMilliseconds</code>	Use to get the milliseconds from a given date according to universal time (UTC).
<code>getUTCMinutes</code>	Use to get the minutes from a given date according to universal time (UTC).

<code>getUTCMonth</code>	Returns the months of given date according to universal time.
<code>getUTCSeconds</code>	Returns the seconds of a given date according to universal time.
<code>getFullYear</code>	Use to get the year of a given date according to local time.
<code>parse</code>	Takes a date string (such as "Jan 21, 1987") and returns the number of milliseconds since January 1, 1970, 00:00:00, local time.
<code>setDate</code>	Use to set the day of the month for a given date according to local time.
<code>setFullYear</code>	Use to set the full year for a given date according to local time.
<code>setHours</code>	Use to set the hours for a given date according to local time.
<code>setMilliseconds</code>	Use to set the milliseconds for a given date according to local time.
<code>setMinutes</code>	Use to set the minutes for a given date according to local time.
<code>setMonth</code>	Use to set the month for a given date according to local time.
<code>setSeconds</code>	Use to set the seconds for a given date according to local time.
<code>setTime</code>	Use to set the value of a Date object according to local time.
<code>setUTCDate</code>	Use to set the numeric day of the month according to Coordinated Universal Time (UTC).
<code>setUTCFullYear</code>	Use to set the full year for a given date according to Coordinated Universal Time (UTC).
<code>setUTCHours</code>	Use to set the hour for a specified date according to Coordinated Universal Time (UTC).
<code>setUTCMilliseconds</code>	Use to set the milliseconds for a specified date according to Coordinated Universal Time (UTC).
<code>setUTCMinutes</code>	Use to set the minutes for a specified date according to Universal Coordinated Time (UTC).
<code>setUTCMonth</code>	Use to set the month for a given date according to local time.
<code>setUTCSeconds</code>	Use to set the seconds for a given date according to universal time.
<code>setYear</code>	Use to set the year for a given date according to local time.

<code>toLocaleString</code>	Use to convert a date to a string, using the current locale time.
<code>toSource</code>	Returns a string which represents the source code of the date object.
<code>toString</code>	Use to convert a date to a string.
<code>toUTCString</code>	Use to convert a date to a string, using the universal time convention.
<code>UTC</code>	Returns the number of milliseconds in a Date object since January 1, 1970, 00:00:00, universal time.
<code>valueOf</code>	Use to get the primitive value of date object.

Example: Get upto seconds for this time.

```
<html>
<head>
  <title>Get date and time in JavaScript.</title>
</head>
<body>
  <h2>Get date and time using JavaScript.</h2>
  <h4>Output after using the basic methods of the Date() class.</h4>
  <div id="date"> </div>
  <div id="time"> </div>
  <script type="text/javascript">
    let date = document.getElementById("date");
    let time = document.getElementById("time");
    // creating the date object and getting the date and time
    let newDate = new Date();
    let year = newDate.getFullYear();
    let month = newDate.getMonth();
    let todaySDate = newDate.getDate();
    let hours = newDate.getHours();
    let minutes = newDate.getMinutes();
    let seconds = newDate.getSeconds();
    date.innerHTML = " " + todaySDate + "/" + month + "/" + year;
    time.innerHTML = hours + ":" + minutes + ":" + seconds;
  </script>
</body>
</html>
```

Number Object

The JavaScript Number object represents numerical data as floating-point numbers. It contains different properties (constants) and methods for working on numbers. In general, you do not need to worry about Number objects because the browser automatically converts number literals to instances of the number class.

Creating Numbers.

Numbers can be created in different ways.

- Using Number literals:
- Using the Number() constructor:

Example:

```
let num1 = 24;
let num2 = 35.65;
const numb = new Number(12345);
const numb1 = new Number("23456");
```

NOTE: If value can't be converted to number, it returns NaN(Not a Number) .

NumberObject Properties.

Name	Description
MAX_VALUE	Returns the maximum numeric value represented in JavaScript.
MIN_VALUE	Returns the number closest to 0, not the negative number represented in JavaScript.
NaN	Represents a value that is not a number
NEGATIVE_INFINITY	A special numeric value represents negative infinity.
POSITIVE_INFINITY	A special numeric value represents positive infinity.

NumberObject Methods.

Name	Description
isFinite()	Checks whether the passed value is a finite number.
isInteger()	Checks whether the passed value is an integer.

<code>isNaN()</code>	Checks whether the passed value is NaN and its type is Number.
<code>isSafeInteger()</code>	Checks whether a value is a safe integer.
<code>toExponential()</code>	Converts a number to exponential notation.
<code>toFixed()</code>	Formats a number using fixed-point notation.
<code>toPrecision()</code>	Returns a string representing the number to the specified precision.
<code>toString()</code>	Converts a number to a string.
<code>valueOf()</code>	Returns the primitive value of a Number object.

Example:

```
// Number Methods in JavaScript
// Checking Number Types and Values
Number.isFinite(10); // true
Number.isFinite(Infinity); // false
Number.isFinite(NaN); // false
Number.isInteger(10); // true
Number.isInteger(10.5); // false
Number.isNaN(NaN); // true
Number.isNaN("hello"); // false (Note: isNaN() converts non-numbers to
numbers first)
Number.isSafeInteger(Math.pow(2, 53) - 1); // true (Max safe integer)
Number.isSafeInteger(Math.pow(2, 53)); // false (Too large)

// Number Formatting
const num = 123.456789;
num.toExponential(); // "1.23456789e+2"
num.toExponential(2); // "1.23e+2"
num.toFixed(); // "123"
num.toFixed(2); // "123.46"
num.toPrecision(); // "123.456789"
num.toPrecision(5); // "123.46"
num.toString(); // "123.456789"

// Value Of
const numObj = new Number(10);
```

```
numObj.valueOf(); // 10
```

Boolean Object

Boolean values can be one of two values: true or false, representing the truth value of a logical proposition. To represent logical values, JavaScript uses the Boolean data type, which has two possible values: true or false. These values often result from comparisons or logical operations. Additionally, the Boolean() function can convert other types of values into Boolean, determining their truthy or falsy nature.

The Boolean() function is used to explicitly convert a value to its Boolean equivalent. Truthy values return true, while falsy values return false.

Creating Boolean.

```
const value = true;
const value = new Boolean(0);
```

Example:

```
console.log(Boolean("Hello")); // true
console.log(Boolean(""));      // false
console.log(Boolean(1));       // true
console.log(Boolean(0));       // false
```

Math Object

The JavaScript Math object is used to perform mathematical tasks. The Math object is a static built-in object, so you won't need to instantiate it, all its properties and methods can be accessed directly.

The JavaScript Math object provides a number of useful properties and methods for performing mathematical tasks like, generating random numbers, rounding numbers, obtaining values such as PI and performing calculation, and so on. It also includes methods for performing mathematical task that are normally impossible or too complex to perform using standard mathematical operators (+, -, *, and /) such as calculating sine or cosine values.

Math Object Properties.

Property	Description
E	Returns Euler's number, the base of natural logarithms, e, approximately 2.718
LN2	Returns the natural logarithm of 2, approximately 0.693

LN10	Returns the natural logarithm of 10, approximately 2.302
LOG2E	Returns the base 2 logarithm of e, approximately 1.442
LOG10E	Returns the base 10 logarithm of e, approximately 0.434
PI	Returns the ratio of the circumference of a circle to its diameter (i.e. π). The approximate value of PI is 3.14159
SQRT1_2	Returns the square root of 1/2, approximately 0.707
SQRT2	Returns the square root of 2, approximately 1.414

Math Object Properties.

Method	Description
abs()	Returns the absolute value of a number.
acos()	Returns the arccosine of a number, in radians.
acosh()	Returns the hyperbolic arccosine of a number.
asin()	Returns the arcsine of a number, in radians
asinh()	Returns the hyperbolic arcsine of a number.
atan()	Returns the arctangent of a number, in radians.
atan2(y, x)	Returns the arctangent of the quotient of its arguments.
atanh()	Returns the hyperbolic arctangent of a number.
cbrt()	Returns the cube root of a number.
ceil()	Returns the next integer greater than or equal to a given number (rounding up).
cos()	Returns the cosine of the specified angle. The angle must be specified in radians.
cosh()	Returns the hyperbolic cosine of a number.

<code>exp(x)</code>	Returns e^x , where x is the argument, and e is Euler's number (also known as Napier's constant), the base of the natural logarithms.
<code>floor()</code>	Returns the next integer less than or equal to a given number (rounding down).
<code>log()</code>	Returns the natural logarithm (base e) of a number.
<code>max(x, y, ...)</code>	Returns the highest-valued number in a list of numbers.
<code>min(x, y, ...)</code>	Returns the lowest-valued number in a list of numbers.
<code>pow(x, y)</code>	Returns the base to the exponent power, that is, x^y .
<code>random()</code>	Returns a random number between 0 and 1 (including 0, but not 1).
<code>round()</code>	Returns the value of a number rounded to the nearest integer.
<code>sin()</code>	Returns the sign of a number (given in radians).
<code>sinh()</code>	Returns the hyperbolic sine of a number.
<code>sqrt()</code>	Returns the square root of a number.
<code>tan()</code>	Returns the tangent of a number.
<code>tanh()</code>	Returns the hyperbolic tangent of a number.
<code>trunc(x)</code>	Returns the integer part of a number by removing any fractional digits.

Example:

```
// Math.abs() - Returns the absolute value of a number
console.log(Math.abs(-5)); // 5
console.log(Math.abs(5)); // 5

// Math.cbrt() - Returns the cube root of a number
console.log(Math.cbrt(27)); // 3
console.log(Math.cbrt(-27)); // -3

// Math.ceil() - Returns the smallest integer greater than or equal to a
number
```

```
console.log(Math.ceil(4.1)); // 5
console.log(Math.ceil(4.9)); // 5

// Math.cos() - Returns the cosine of a number (in radians)
console.log(Math.cos(0)); // 1
console.log(Math.cos(Math.PI / 2)); // 0

// Math.exp() - Returns e raised to the power of a number
console.log(Math.exp(1)); // 2.718281828459045
console.log(Math.exp(0)); // 1

// Math.floor() - Returns the largest integer less than or equal to a
number
console.log(Math.floor(4.9)); // 4
console.log(Math.floor(4.1)); // 4

// Math.log() - Returns the natural logarithm (base e) of a number
console.log(Math.log(1)); // 0
console.log(Math.log(Math.E)); // 1

// Math.max() - Returns the largest of zero or more numbers
console.log(Math.max(1, 2, 3)); // 3
console.log(Math.max(-1, -2, -3)); // -1

// Math.min() - Returns the smallest of zero or more numbers
console.log(Math.min(1, 2, 3)); // 1
console.log(Math.min(-1, -2, -3)); // -3

// Math.pow(x, y) - Returns the base raised to the exponent power
console.log(Math.pow(2, 3)); // 8
console.log(Math.pow(3, 2)); // 9

// Math.random() - Returns a random number between 0 (inclusive) and 1
(exclusive)
console.log(Math.random()); // Random value between 0 and 1

// Math.round() - Returns the value of a number rounded to the nearest
integer
console.log(Math.round(4.5)); // 5
console.log(Math.round(4.4)); // 4

// Math.sin() - Returns the sine of a number (in radians)
```

```

console.log(Math.sin(0));    // 0
console.log(Math.sin(Math.PI / 2)); // 1

// Math.sqrt() - Returns the square root of a number
console.log(Math.sqrt(16)); // 4
console.log(Math.sqrt(9));  // 3

// Math.trunc(x) - Returns the integer part of a number by removing any
fractional digits
console.log(Math.trunc(4.9)); // 4
console.log(Math.trunc(-4.9)); // -4

```

Regular Expression

A Regular Expression (or **Regex**) is a powerful tool used for searching, matching, and manipulating text based on specific patterns. It allows you to perform complex text operations like finding substrings, replacing text, or validating input using a concise and flexible syntax.

Regex is commonly used for:

- **Pattern matching:** Finding specific patterns in text.
- **Validation:** Ensuring that input conforms to a specific format (e.g., validating email addresses or phone numbers).
- **Text manipulation:** Replacing parts of strings or extracting certain parts

As other javascript Objects, Regex object can also be created by using literal and constructor. When creating using literal, pattern should be enclosed between slashes.

```

const re = /ab+c/;
const re = new RegExp("ab+c");

```

Regular expressions are used with the RegExp methods **test()** and **exec()** and with the String methods **match()**, **matchAll()**, **replace()**, **replaceAll()**, **search()**, and **split()**.

Example: Testing regex whether string is valid email or not.

```

const regex = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
console.log(regex.test("test@example.com")); // true
console.log(regex.test("invalid-email.com")); // false

```

Example: Extracting All Numbers from a String

```
const text = "I have 5 pens and 7 pencils .";  
const regex = /\d+/g; // Matches one or more digits globally  
console.log(text.match(regex)); // ["5", "7"]
```

If we had used, `/\d+/`, it would match only first occurrence but not globally.

DOM Object.

The Document Object Model gives you scripting access to all the elements on a web page. Using JavaScript, you can create, modify and remove elements in the page dynamically.

"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."

The W3C DOM standard is separated into 3 different parts:

- Core DOM - standard model for all document types
- XML DOM - standard model for XML documents
- HTML DOM - standard model for HTML documents

One of the most important objects in client-side JavaScript programming is the Document object—which represents the HTML document that is displayed in a browser window or tab. The Window object has a document property, which is reference to a Document object and is also available as a global variable. The Document object represents the content of the window and is the main object of Document Object Model.

The DOM represents a HTML document as a tree structure of objects.

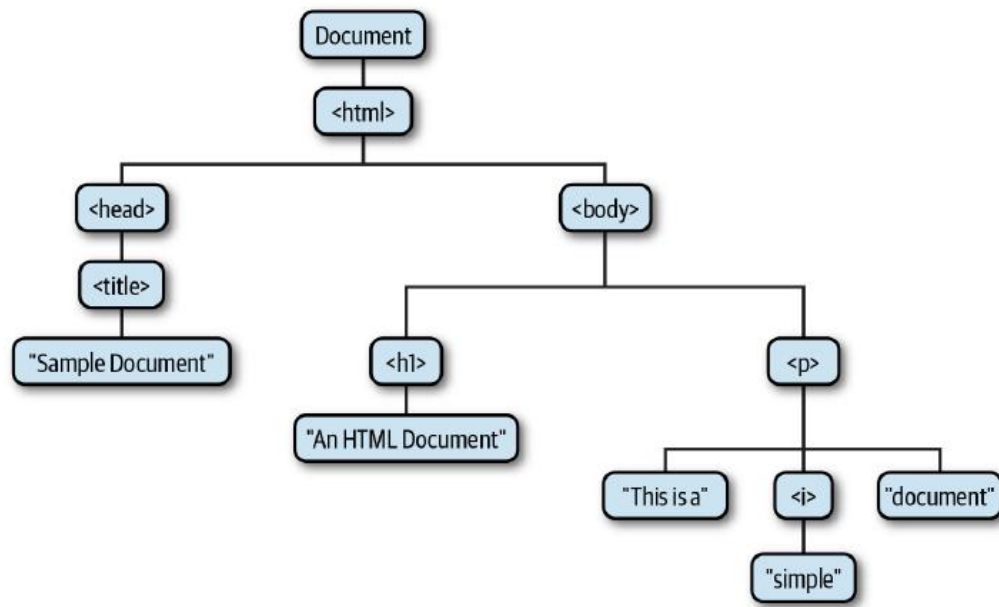
- Each node represents an element in the document.
- Elements of the document can be added, removed and modified.

Let us take example

```
<html>  
<head>  
  <title>Sample Document</title>  
</head>  
<body>
```

```
<h1>An HTML Document</h1>
<p>This is a <i>simple</i> document.</p>
</body>
</html>
```

The DOM representation of above html document is the tree pictured below:



DOM Manipulation

DOM (Document Object Model) manipulation allows you to interact with and modify web pages.

Selecting Elements

- **document.getElementById()** : You can select an element using its unique ID. This method returns Element object.
- **document.getElementsByTagName()**: You can select all elements with specific name using the getElementsByTagName() method of the HTMLDocument object type. This method returns NodeList object, which is like a read-only array of Element objects.
- **document.getElementsByTagName()**: HTML elements can be selected by their tag type using getElementsByTagName() method of the Document object type. This method returns a NodeList object containing found elements in the order in which they appear in the document.

- **document.getElementsByClassName():** To select all elements with a given class or classes use `getElementsByClassName()` method and specify required classes separated by space. It returns `NodeList` object.
- **document.querySelector():** Selects the first element that matches the given CSS selector.
- **document.querySelectorAll():** Selects all elements that match the given CSS selector.

Changing/Retrieving Content

- **element.innerHTML:** Sets or returns the HTML content
- **element.textContent:** Sets or returns the text content

`getElementById:`

```
<!DOCTYPE html>
<html>
<head>
<title>getElementById Example</title>
</head>
<body>
  <div id="myDiv">This is my div.</div>
  <div id="result"></div>
  <script>
    const myDiv = document.getElementById("myDiv");
    const resultDiv = document.getElementById("result");
    resultDiv.innerHTML = "getElementById: " + myDiv.textContent;
  </script>
</body>
</html>
```

`getElementByName`

```
<!DOCTYPE html>
<html>
<head>
<title>getElementsByName Example</title>
</head>
<body>
  <input type="radio" name="gender" value="male"> Male
  <input type="radio" name="gender" value="female"> Female
  <div id="result"></div>
  <script>
```

```

    const genderInputs = document.getElementsByName("gender");
    const resultDiv = document.getElementById("result");
    let output = "getElementsByName: ";
    genderInputs.forEach(input => {
        output += input.value + ", ";
    });
    resultDiv.innerHTML = output.slice(0, -2); // Remove the trailing
comma and space
</script>
</body>
</html>

```

getElementsByTagName

```

<!DOCTYPE html>
<html>
<head>
<title>getElementsByTagName Example</title>
</head>
<body>
    <p>Paragraph 1</p>
    <p>Paragraph 2</p>
    <div id="result"></div>
    <script>
        const paragraphs = document.getElementsByTagName("p");
        const resultDiv = document.getElementById("result");
        resultDiv.innerHTML = "getElementsByTagName (length): " +
paragraphs.length + "<br>";
        resultDiv.innerHTML += "getElementsByTagName (first p): " +
paragraphs[0].textContent;
    </script>
</body>
</html>

```

getElementsByClassName

```

<!DOCTYPE html>
<html>
<head>
<title>getElementsByClassName Example</title>
</head>
<body>
    <div class="highlight">Item 1</div>

```



```

<div class="highlight">Item 2</div>
<div id="result"></div>
<script>
    const highlightedItems = document.getElementsByClassName("highlight");
    const resultDiv = document.getElementById("result");
    resultDiv.innerHTML = "getElementsByClassName (length): " +
highlightedItems.length;
</script>
</body>
</html>

```

querySelector

```

<!DOCTYPE html>
<html>
<head>
<title>querySelector Example</title>
</head>
<body>
    <div id="myDiv" class="highlight">This is my div.</div>
    <div id="result"></div>
    <script>
        const myDiv = document.querySelector("#myDiv.highlight");
        const resultDiv = document.getElementById("result");
        resultDiv.innerHTML = "querySelector: " + myDiv.textContent;
    </script>
</body>
</html>

```

querySelectorAll

```

<!DOCTYPE html>
<html>
<head>
<title>querySelectorAll Example</title>
</head>
<body>
    <p class="paragraph">Paragraph 1</p>
    <p class="paragraph">Paragraph 2</p>
    <div id="result"></div>
    <script>
        const paragraphs = document.querySelectorAll("p.paragraph");
        const resultDiv = document.getElementById("result");

```

```

    let output = "querySelectorAll (length): " + paragraphs.length +
"<br>";
    paragraphs.forEach(p => output += "querySelectorAll (all p): " +
p.textContent + "<br>");
    resultDiv.innerHTML = output;
</script>
</body>
</html>

```

Event Handling

Events are actions or occurrences that happen in the browser, which the browser can detect and respond to. These can be triggered by user interactions such as clicking a button, typing in a text field, or moving the mouse, as well as by other actions like loading a webpage or resizing the browser window.

js react over these events and allow the execution. This process of reacting over the events is called Event Handling.

JavaScript supports a variety of event types. Common categories include:

- **Mouse Events:** click, dblclick, mousemove, mouseover, mouseout
- **Keyboard Events:** keydown, keypress, keyup
- **Form Events:** submit, change, focus, blur
- **Window Events:** load, resize, scroll

An event can be handled by one or more event handlers. If an event has multiple event handlers, all the event handlers will be executed when the event is fired.

There are three ways to assign event handlers.

1. By using HTML event handler attribute:

```

<button onclick="alert('Button clicked!')">Click Me</button>

<input type="button" value="Save" onclick="handleClick()">
<script>
    function handleClick() {
        alert('Clicked!');
    }
</script>

```

2. DOM Property Handlers/ DOM event handler

```
<input type="button" id="btn">

let btn = document.querySelector('#btn');
btn.onclick = function() {
    alert(this.id);
};
```

This can be written alternatively as

```
btn.onclick = () => {
    alert(this.id);
};
```

To remove the event handler, you set the value of the event handler property to null:

```
btn.onclick = null;
```

3. addEventListener()

```
<input type="button" id="btn">

let btn = document.getElementById("btn");

btn.addEventListener("click", () => {
    alert("Button clicked using addEventListener!");
});

btn.addEventListener('click',function(event) {
    alert(event.type); // click
});

btn.addEventListener('click',function(event) {
    alert('Button clicked using addEventListener!');
});

// add the event listener
let handleClick = function() {
    alert('Clicked!');
};
btn.addEventListener('click', handleClick);
```

To remove the event handler, you call removeEventListener method

```
btn.removeEventListener('click', handleClick);
```

For the event added via anonymous methods can not be removed.

Form Validation

Form Validation refers to the process of validating user input data in html form on the client-side, typically using JavaScript, before sending it to the server for further processing. The goal of this validation is to provide quick feedback to the user regarding input errors, reducing the number of server-side requests and improving the overall user experience.

JavaScript provides a way to validate form's data on the client's computer before sending it to the web server. Form validation generally performs two functions.

- **Basic Validation** – First of all, the form must be checked to make sure all the mandatory fields are filled in. It would require just a loop through each field in the form and check for data.
- **Data Format Validation** – Secondly, the data that is entered must be checked for correct form and value. Your code must include appropriate logic to test correctness of data.

Basic Validation Example:

```
<html>
  <head>
    <title>Form Validation</title>
    <script type="text/javascript">
      function validate() {
        var name = document.myForm.Name.value;
        var email = document.myForm.EMail.value;
        var zip = document.myForm.Zip.value;
        var country = document.myForm.Country.value;

        if (name == "" || email == "" || zip == "" || country == "1"){
          alert("Please fill all fields.");
          return false;
        }
        return true;
      }
    </script>
  </head>
```

```

<body>
  <form action="postdata.php" method="POST" name="myForm"
onsubmit="return(validate());">

    <div style="margin-bottom: 10px;">
      <label for="Name">Name</label>
      <input type="text" name="Name" id="Name" />
    </div>

    <div style="margin-bottom: 10px;">
      <label for="EMail">EMail</label>
      <input type="text" name="EMail" id="EMail" />
    </div>

    <div style="margin-bottom: 10px;">
      <label for="Zip">Zip Code</label>
      <input type="text" name="Zip" id="Zip" />
    </div>

    <div style="margin-bottom: 10px;">
      <label for="Country">Country</label>
      <select name="Country" id="Country">
        <option value="-1" selected>[choose yours]</option>
        <option value="1">USA</option>
        <option value="2">UK</option>
        <option value="3">INDIA</option>
      </select>
    </div>

    <div style="margin-bottom: 10px;">
      <input type="submit" value="Submit" />
    </div>

  </form>
</body>
</html>

```

Data Validation Example

Let us validate email field by checking whether the input email is in correct format or not. An email address must contain at least a '@' sign and a dot (.). Also, the '@' must not be the

first character of the email address, and the last dot must at least be one character after the '@' sign.

```
function validateEmail() {
    var emailID = document.getElementById("EMail").value;
    atpos = emailID.indexOf("@");
    dotpos = emailID.lastIndexOf(".");

    if (atpos < 1 || ( dotpos - atpos < 2 )) {
        alert("Please enter correct email ID")
        document.myForm.EMail.focus() ;
        return false;
    }
    return( true );
}
```

Alternatively we can use regular expression to check the proper format of email.

```
function validateEmail() {
    var emailID = document.getElementById("EMail").value;

    // Regular expression to check for the required conditions
    var emailRegex = /^[A-Za-z0-9._-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;

    // Test the email with regex
    if (!emailRegex.test(emailID)) {
        alert("Please enter correct email ID");
        document.myForm.EMail.focus();
        return false;
    }
    return true;
}
```

To Combine basic validation with data validation, we call data validation after basic validation is done.

Example:

```
function validate() {
    var name = document.myForm.Name.value;
    var email = document.getElementById("EMail").value;
    var zip = document.getElementById("Zip").value;
    var country = document.getElementById("Country").value;
```

```

        if (name == "" || email == "" || zip == "" || country == "-
1"){
            alert("Please fill all fields.");
            return false;
        }
        return validateEmail();
    }
}

```

Error Handling

Error handling in JavaScript is a process to detect and handle errors that occurs during the execution of a program. Errors can be a **syntax**, **runtime** or **logical errors**. An error occurred during the execution of the program is called a runtime error or an exception.

In JavaScript, errors can occur due to programming mistakes, incorrect user input, etc. Errors can disrupt code execution and lead to bad user experience. Effective error & exception handling is required for building robust, reliable and user friendly applications in JavaScript.

Exception Handling Errors Using try...catch

- **Try Block:** You place code that may potentially throw an error here.
- **Catch Block:** This block executes if an error occurs in the try block.
- **finally Block:** The Code inside this block will always run, whether an error occurred or not.
- **throw:** Used to create custom errors and stop code execution.

Syntax

```

try {
    // Code that may throw an error
} catch (error) {
    // Code to handle the error
} finally {
    // Optional: Code that runs regardless of success or error
}

```

Example: Here is an example where we are trying to call a **non-existing** function which in turn is raising an exception.

```

<html>
<head>
<script>
    try {
        var a = 100;
    }
}

```

```

        alert(myFunc(a));
    }
    catch (e) {
        alert(e);
    }
    finally {
        alert("Finally block will always execute!" );
    }
</script>
</head>
<body>
    <p>Exception handling using try...catch...finally statements</p>
</body>
</html>

```

Example for divide by zero exception

```

try {
    let res = 10 / 0;
    if (!isFinite(res)) {
        throw new Error("Cannot divide by zero");
    }
    console.log(res);
} catch (error) {
    console.error("Error occurred:", error.message);
} finally {
    console.log("Execution completed");
}

```

Handling Cookies

Cookies are small strings of data that are stored directly in the browser. They are a part of the HTTP protocol. A cookie is a small piece of data that is stored on the user's computer by the web browser. Cookies are often used to store user preferences or other data that needs to persist across multiple sessions or pages on a website.

It's important to understand that HTTP is stateless. When you send two subsequent HTTP requests to the server, there is no link between them. In other words, the server cannot know if the two requests are from the same web browser. Therefore, a cookie is used to tell if the two requests came from the same web browser. After cookie is set for some site/domain, **Whenever a user sends a request to the server, the cookie is added with that request automatically. Due to the cookie, the server recognizes the users.**

In practice, cookies serve the following purposes:

- **Session management** – cookies allow you to manage any information that the server should remember. For example, logins, shopping carts, etc.
- **Personalization** – cookies allow you to store user preferences, themes, and setting specific to a user.
- **Tracking** – cookies help record and analyze user behaviors in advertising.

The advisable maximum size of a cookie is 4KB and 50 cookies per domain. So for single domain, there could be one single cookie with size of 4KB or within 50 cookies where sum of all less than 4KB.

In JavaScript, we can create, read, update and delete a cookie by using **document.cookie** property.

Creating/Changing Cookies

With JavaScript, you can create or change a cookie in the same way. If the cookie exists, its value will be changed and if it does not exist, the cookie will be created.

```
document.cookie = "username=John Doe";
```

Reading Cookies

```
let x = document.cookie;
```

document.cookie will return all cookies in one string much like:

```
cookie1=value; cookie2=value; cookie3=value;
```

Example

Create Cookie:

```
<!DOCTYPE html>
<html>
  <head> </head>
  <body>
    <input type="button" value="setCookie" onclick="setCookie()" />
    <input type="button" value="getCookie" onclick="getCookie()" />
    <script>
      function setCookie() {
        document.cookie = "username=John Doe";
      }
      function getCookie() {
```

```

        if (document.cookie.length != 0) {
            alert(document.cookie);
        } else {
            alert("Cookie not available");
        }
    }
</script>
</body>
</html>

```

Example

In this example, we provide choices of color and pass the selected color value to the cookie. Now, cookie stores the last choice of a user in a browser. So, on reloading the web page, the user's last choice will be shown on the screen

```

<!DOCTYPE html>
<html>
  <head> </head>
  <body>
    <select id="color" onchange="display()">
      <option value="Select Color">Select Color</option>
      <option value="yellow">Yellow</option>
      <option value="green">Green</option>
      <option value="red">Red</option>
    </select>
    <script>
      if (document.cookie.length != 0) {
        var array = document.cookie.split("=");
        document.getElementById("color").value = array[1];
        document.body.style.background = array[1];
      }

      function display() {
        var value = document.getElementById("color").value;
        if (value != "Select Color") {
          document.body.style.background= value;
          document.cookie = "color=" + value;
        }
      }
    </script>
  </body>
</html>

```

```

    </script>
  </body>
</html>

```

Cookie Properties

Attribute	Description
expires	It maintains the state of a cookie up to the specified date and time.
max-age	It maintains the state of a cookie up to the specified time. Here, time is given in seconds.
path	It expands the scope of the cookie to the pages of a website. If a cookie has path=/blog, the cookie will only be sent to the server when the URL is something like www.example.com/blog or any sub-page of /blog, such as www.example.com/blog/post1. If the cookie has path=/, it will be available for the entire domain, meaning it will be sent with requests to all pages on the domain.
domain	It is used to specify the domain for which the cookie is valid.

Examples:

```
document.cookie = "username=John Doe; expires=Thu, 28 Feb 2025 12:00:00 UTC";
```

```
document.cookie = "username=John Doe; max-age=3600"; // Expires in 1 hour
document.cookie = "username=John Doe; path=/"; // Cookie is accessible across the entire domain
```

```
document.cookie = "username=John Doe; domain=.example.com"; // Cookie is available for all subdomains of example.com
```

Deleting Cookies

To delete a cookie in JavaScript, you essentially set the cookie's expiration date to a time in the past. This will make the browser remove the cookie. But ensure that the path and domain match the original cookie's settings.

```
document.cookie = "username=; max-age=0; path=/";
```

JQuery

jQuery is a fast, small, and feature-rich JavaScript library. It makes things like HTML document traversal and manipulation, event handling, animation, and Ajax much simpler with an easy-to-use API that works across a multitude of browsers. With a combination of versatility and extensibility, jQuery has changed the way that millions of people write JavaScript.

jQuery also gives you the ability to create an Ajax based application in a quick and simple way. Big companies like Google, Microsoft and IBM are using the jQuery for their applications.

Important core features supported by jQuery

- **DOM manipulation** – The jQuery made it easy to select DOM elements, negotiate them and modifying their content by using cross-browser open source selector engine called **Sizzle**.
- **Event handling** – The jQuery offers an elegant way to capture a wide variety of events, such as a user clicking on a link, without the need to clutter the HTML code itself with event handlers.
- **AJAX Support** – The jQuery helps you a lot to develop a responsive and feature rich site using AJAX technology.
- **Animations** – The jQuery comes with plenty of built-in animation effects which you can use in your websites.
- **Cross Browser Support** – The jQuery has cross-browser support, and works well in IE 6.0+, FF 2.0+, Safari 3.0+, Chrome and Opera 9.0+
- **Latest Technology** – The jQuery supports CSS3 selectors and basic XPath syntax

Advantages of using JQuery

If you're not familiar with jQuery, you might be wondering what makes jQuery so special. There are several advantages why one should opt for jQuery:

- **Save lots of time** — You can save lots of time and efforts by using the jQuery inbuilt effects and selectors and concentrate on other development work.
- **Simplify common JavaScript tasks** — jQuery considerably simplifies the common JavaScript tasks. Now you can easily create feature rich and interactive web pages with fewer lines of codes, a typical example is implementing Ajax to update the content of a page without refreshing it.

- **Easy to use** — jQuery is very easy to use. Anybody with the basic working knowledge of HTML, CSS and JavaScript can start development with jQuery.
- **Compatible with browsers** — jQuery is created with modern browsers in mind and it is compatible with all major modern browsers such as Chrome, Firefox, Safari, Internet Explorer, etc.
- **Lightweight** – The jQuery is very lightweight library - about 19KB in size (Minified and gzipped).
- **Absolutely Free** — And the best part is, it is completely free to download and use.

Using JQuery in HTML pages

We can use JQuery in HTML in one of the following ways .

1. Including jQuery.js

```
<script src="jquery.js"></script>
<script>
    $(document).ready(function () {
        alert("Test");
    });
</script>
```

2. By using jQuery CDN.

If you don't want to download and host jQuery yourself, you can include it from a CDN (Content Delivery Network).

```
<head>
    <script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
</head>
```

NOTE : min is minified version. CDN location can be different but try to use official one.

"https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"

Basic jQuery Syntax

A jQuery statement typically starts with the dollar sign (\$) and ends with a semicolon (;). In jQuery, the dollar sign (\$) is just an alias for jQuery.

```
<script>
    $(document).ready(function(){
        // Some code to be executed...
        alert("Hello World!");
    });
</script>
```

OR

```
<script>
    $(function(){
        // Some code to be executed...
        alert("Hello World!");
    });
</script>
```

[Both of the above scripts are equivalent; we can use any one of these. But for simplicity of understanding , the first option is preferred.]

- The `<script>` element — Since jQuery is just a JavaScript library, so the jQuery code can be placed inside the `<script>` element. However, if you want to place it in an external JavaScript file, which is preferred, you just remove this part.
- The `$(document).ready(handler);` — This statement is typically known as ready event. Where the handler is basically a function that is passed to the `ready()` method to be executed safely as soon as the document is ready to be manipulated i.e. when the DOM hierarchy has been fully constructed. I.e. When the page is just ready to be rendered.

jQuery Selectors

The ability of making the DOM elements selection simple and easy is one of the most powerful feature of the jQuery.

jQuery selectors allow you to select and manipulate HTML element(s). jQuery selectors are used to "find" (or select) HTML elements based on their name, id, classes, types, attributes, values of attributes and much more. It's based on the existing CSS Selectors, and in addition, it has some own custom selectors.

All selectors in jQuery start with the dollar sign and parentheses: `$()`.

The jQuery supports almost all the selectors defined in the latest CSS3 specifications, as well as it has its own custom selectors. These custom selectors greatly enhance the capabilities selecting the HTML elements on a page.

Selector	Description	Example
*	Selects all elements on the page.	<code>\$('div *')</code> selects all elements inside a div.
#id	Selects an element with the given id.	<code>\$('#myDiv')</code> selects the element with <code>id="myDiv"</code> .
.class	Selects all elements with the given class.	<code>\$('.myClass')</code> selects all elements with <code>class="myClass"</code> .
element	Selects all elements of the specified type.	<code>\$('p')</code> selects all <code><p></code> elements.
element, element	Selects all elements of the specified types.	<code>\$('p, div')</code> selects all <code><p></code> and <code><div></code> elements.
#id > element	Selects direct child elements of the specified id.	<code>\$('#parent > p')</code> selects all <code><p></code> elements directly inside <code>#parent</code> .
element[attr]	Selects elements with a specified attribute.	<code>\$('input[type="text"]')</code> selects all input elements of type text.
element[attr="value"]	Selects elements with a specific attribute value.	<code>\$('a[href="https://example.com"]')</code> selects all links with <code>href="https://example.com"</code> .
:first	Selects the first element in the matched set.	<code>\$('li:first')</code> selects the first <code></code> element.
:last	Selects the last element in the matched set.	<code>\$('li:last')</code> selects the last <code></code> element.
:eq(index)	Selects the element at the specified index (0-based).	<code>\$('li:eq(2)')</code> selects the third <code></code> element.
:even	Selects elements with an even index.	<code>\$('li:even')</code> selects all even-indexed <code></code> elements.

Selector	Description	Example
:odd	Selects elements with an odd index.	\$('li:odd') selects all odd-indexed elements.
:not(selector)	Selects all elements except those matching the selector.	\$('p:not(.intro)') selects all <p> elements except those with class="intro".
:contains(text)	Selects elements that contain the specified text.	\$('p:contains("Hello")') selects all <p> elements containing the text "Hello".
:hidden	Selects elements that are hidden.	\$('div:hidden') selects all hidden <div> elements.
:visible	Selects elements that are visible.	\$('div:visible') selects all visible <div> elements.
:focus	Selects the currently focused element.	\$('input:focus') selects the input field that is currently focused.
:enabled	Selects enabled form elements.	\$('input:enabled') selects all enabled input fields.
:disabled	Selects disabled form elements.	\$('input:disabled') selects all disabled input fields.
:checked	Selects checked checkboxes or radio buttons.	\$('input:checked') selects all checked input elements.
:selected	Selects selected option elements in a dropdown.	\$('option:selected') selects all selected <option> elements.
:parent	Selects elements that have at least one child.	\$('div:parent') selects all <div> elements with children.
:empty	Selects elements that have no children.	\$('div:empty') selects all empty <div> elements.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>jQuery Select Element by ID</title>
<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
<script>
$(document).ready(function(){
    // Highlight element with id mark
    $("#mark").css("background", "yellow");
});

</script>
</head>
<body>
    <p id="mark">This is a paragraph.</p>
    <p>This is another paragraph.</p>
    <p>This is one more paragraph.</p>
    <p><strong>Note:</strong> The value of the id attribute must be unique
in an HTML document.</p>
</body>
</html>
```

Example2:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>jQuery Select Element by Class</title>
<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
<script>
$(document).ready(function(){
    // Highlight elements with class mark
    $(".mark").css("background", "yellow");
});

</script>
</head>
<body>
    <p class="mark">This is a paragraph.</p>
```

```
<p class="mark">This is another paragraph.</p>
<p>This is one more paragraph.</p>
</body>
</html>
```

jQuery Events

In general, the events can be categorized into four main groups

- mouse events,
- keyboard events,
- form events and
- document/window events.

Mouse Events

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Executing a Function on Click Event in jQuery</title>
<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
<style>

    p.highlight{
        background: yellow;
    }
    p.enter{
        background: red;
    }
</style>
<script>
$(document).ready(function(){
    $("p").click(function(){
        $(this).slideUp();
    });
    $("p").dblclick(function(){
        $(this).slideUp();
    });
    $("p").hover(function(){
```

```

        $(this).addClass("highlight");
    }, function(){
        $(this).removeClass("highlight");
    });
    $("p").mouseenter(function(){
        $(this).addClass("enter");
    });
    $("p").mouseleave(function(){
        $(this).removeClass("enter");
    });
});
</script>
</head>
<body>
    <p>Click on me and I'll disappear.</p>
    <p>Click on me and I'll disappear.</p>
    <p>Click on me and I'll disappear.</p>
</body>
</html>

```

Other all the events are like the javascript events. The most used document/window event is `ready()` method. It specifies a function to execute when the DOM is fully loaded.

jQuery Effects

jQuery provides a wide range of built-in effects and animations to easily manipulate HTML elements. Here are some common jQuery effects:

Show/Hide

These effects are used to display or hide elements, typically with an animation.

Show: Displays an element.

```
$('#element').show();
```

Hide: Hides an element.

```
$('#element').hide();
```

Toggle: Switches between showing and hiding an element.

```
$('#element').toggle();
```

Fade In: Gradually makes an element visible.

```
$('#element').fadeIn();
```

Fade Out: Gradually hides an element.

```
$('#element').fadeOut();
```

Fade Toggle: Toggles the visibility of an element with a fade effect.

```
$('#element').fadeToggle();
```

Slide

These effects are used to make elements slide in and out of view.

Slide Down: Gradually slides an element down to make it visible.

```
$('#element').slideDown();
```

Slide Up: Gradually slides an element up to hide it.

```
$('#element').slideUp();
```

Slide Toggle: Toggles between slide down and slide up.

```
$('#element').slideToggle();
```

Animation

With `animate()`, you can create custom animations by changing CSS properties.

```
$('#element').animate({  
  opacity: 0.5,  
  height: '200px',  
}, 1000); // 1000 is the duration in milliseconds
```

You can animate many properties like width, height, left, top, opacity, etc.

Chaining

In jQuery, chaining refers to the ability to call multiple methods on the same element or set of elements in a single line of code. This is possible because most jQuery methods return the jQuery object itself, allowing you to chain other methods together.

```
$('#element')  
  .fadeOut(500) // Fade out over 500ms  
  .fadeIn(500)  // Fade in over 500ms  
  .slideUp(500) // Slide up over 500ms
```

```
.slideDown(500); // Slide down over 500ms
```

Callbacks

In jQuery, callbacks are functions that you pass to jQuery methods to be executed once an effect, animation, or action has completed. They are useful for triggering additional code after an effect or animation has finished. This allows you to create smooth, sequential interactions on your webpage.

```
$('#element').fadeOut(500, function()  
{  
  alert('Fade out is complete!');  
});
```

Example: For fading in and out

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>jQuery Fade Example</title>  
  <script src="jquery-3.7.1.min.js"></script>  
  <style>  
    #myElement {  
      width: 200px;  
      height: 100px;  
      background-color: lightblue;  
      color: white;  
    }  
  </style>  
</head>  
<body>  
  
  <div id="myElement">Hello, Fade Me!</div>  
  <button id="fadeButton">Fade Out and Fade In</button>  
  
  <script>  
    // jQuery to handle the fade effect  
    $(document).ready(function() {  
      $('#fadeButton').click(function() {  
        $('#myElement')  
          .fadeOut(1000) // Fade out over 1 second  
          .fadeIn(1000); // Fade in over 1 second  
      });  
    });  
  </script>  
</body>  
</html>
```

```

        });
    });
</script>

</body>
</html>

```

NOTE: Normally jQuery callback is used in effects and animations but it can be written and used in javascript as well anywhere according to your requirement. When you want a feature like calling some function after one function is executed, callback can be used.

```

<!DOCTYPE html>
<html>

    <head>
        <script src="jquery-3.7.1.min.js"></script>
    </head>

    <body>
        <input type="button" value="Click" id="mybtn" />
    </body>

    <script>
        $(document).ready(function () {

            $("#mybtn").click(function () {
                console.log("my button click event executed!");
                myFunction(function () {
                    console.log("Callback executed!");
                });
            });

            function myFunction(call) {
                console.log("Normal processing when my function is called");
                call();
            }
        });
    </script>
</html>

```

JSON

JSON or JavaScript Object Notation is a lightweight text-based open standard designed for human-readable data interchange. The JSON format was originally specified by Douglas Crockford, and is described in RFC 4627. The official Internet media type for JSON is application/json. The JSON filename extension is .json. The values are stored in Key, Value pairs.

In JSON, values must be one of the following data types:

- a string
- a number
- an object (JSON object)
- an array
- a boolean
- null

Uses of JSON

- It is used while writing JavaScript based applications that includes browser extensions and websites.
- JSON format is used for serializing and transmitting structured data over network connection.
- It is primarily used to transmit data between a server and web applications.
- Web services and APIs use JSON format to provide public data.
- It can be used with modern programming languages.

JSON Example

```
{  
  "myString": "Hello World",  
  "myInt": 123,  
  "myBoolean": true,  
  "myDecimal": 12.3,
```

```

"myDateTime1": "2019-09-23T10:10:00",
"myDateTime2": "2019-09-23",
"myStringList": ["apple", "banana", "orange"],
"myDictionary": {
  "person1": {
    "id": 201,
    "name": "C#"
  },
  "person2": {
    "id": 302,
    "name": "F#"
  }
},
"myAnotherModel": {
  "myString": "Hi there",
  "myInt": 456
}
}

```

Serialization and Deserialization of JSON (Stringifying and Parsing of JSON)

JSON is a format that encodes objects in a string. [Serialization means to convert an object into the string, and deserialization is its inverse operation \(convert string -> object\).](#)

When transmitting data or storing them in a file, the data are required to be byte strings, but complex objects are seldom in this format. Serialization can convert these complex objects into byte strings for such use. After the byte strings are transmitted, the receiver will have to recover the original object from the byte string. This is known as deserialization.

Serialization : `JSON.stringify()`

Deserialization/Parsing: `JSON.parse()`

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JSON Serialization and Deserialization</title>
</head>
<body>
  <h1>JSON Serialization and Deserialization Example</h1>

```



```

<script>
  // Original JavaScript object
  const originalObject = {
    name: "Ram Shah",
    age: 30,
    isActive: true,
    hobbies: ["reading", "traveling", "coding"],
    address: {
      tole: "Patan Dhoka",
      city: "Lalitpur"
    }
  };

  // 1. Serialization: Convert JavaScript object to JSON string
  const jsonString = JSON.stringify(originalObject);

  console.log("Serialized JSON String:");
  console.log(jsonString);

  // 2. Deserialization: Convert JSON string back to JavaScript object
  const deserializedObject = JSON.parse(jsonString);

  console.log("Deserialized JavaScript Object:");
  console.log(deserializedObject);

  // Accessing properties of the deserialized object
  console.log("Name from deserialized object:", deserializedObject.name);
  // Ram Shah
  console.log("Hobbies from deserialized object:",
deserializedObject.hobbies); // ["reading", "traveling", "coding"]
  console.log("Address from deserialized object:",
deserializedObject.address); // ["reading", "traveling", "coding"]
</script>

</body>
</html>

```